Στέφανος Κάππας 20/15 Ανάκτηση πληροφορίας και μηχανές αναζήτησης Τελική εργασία

Πρόλογος

Η εργασία αυτή είναι η τελική για την εξέταση του Ιανουαρίου 2021. Θα προσπαθείσω να γράψω τον κώδικα και να τον περιγράψω σε αυτό το αρχείο όσο πιο ξεκάθαρα μπορώ. Μπορεί να μην πετύχω το βέλτιστο ή σωστό αποτέλεσμα αλλά η προσπάθειά μου και η διεκπεραίωση κάποιων ή όλων των διαδικασιών μετράει. Η εργασία χωρίζεται σε 4 τμήματα. Η αρχικοποίηση, το συνεργατικό φιλτράρισμα, η πρόβλεψη βαθμολογιών τους και η σύγκριση των μεθόδων σχετικά με ποιο συνεργατικό φιλτράρισμα με πια πρόβλεψη βγάζει το καλύτερο αποτέλεσμα. Ελπίζω να καταφέρω τουλάχιστον την αρχικοποίηση, αλλά εάν μπορώ να συνεχίσω και άλλο θα συνεχίσω μέχρι τέλους. Επίσης η εργασία περιλαμβάνει 2 γλώσσες. Την ελληνική και αραβική γλώσσα.

Μετά από ενάμισι χρόνο και επειδή αυτή εργασία δεν ήταν πλήρης υλοποιημένη παρόλο που πέρασα σε αυτό το μάθημα. Εγώ αποφάσισα να συνεχίσω την υλοποίηση της εργασίας μέχρι τέλος για να εξασκηθώ πρώτον στον προγραμματισμό και δεύτερον την ολοκλήρωση της εργασίας. Έτσι είδα όλες τις μεθόδους εξονυχιστικά από την αρχή, εντόπισα όλα τα σφάλματα που αντιμετώπιζαν οι καθε μία μέθοδος και επίλυσα όλα τα προβλήματα. Επιπλέον πρόσθεσα και άλλες μεθόδους για την ολοκλήρωση του προγράμματος.

Περιεχόμενα

Υπόμνημα Και Παρατηρήσεις ..... σελ. 0
Αρχικοποίηση Α' Ερώτημα ..... σελ. 1
Συνεργαγικό Φιλτράρισμα Β' Ερώτημα ..... σελ. 5
Πρόβλεψη Βαθμολογιών Β' Ερώτημα ..... σελ. 9
Πρόβλεψη Βαθμολογιών Ολοκληρωμένο Γ' Ερώτημα ..... σελ. 12
Πειράματα Που Έκανα Και Σύγκριση Των Μεθόδων ..... σελ. 15
Πίνακες Και Κώδικες ..... σελ. 18

Υπόμνημα Και Παρατηρήσεις

Όλα τα παρακάτω βρίσκονται σε εισαγωγικά "":
Κόκκινο χρώμα σημαίνει σύνδεσμος π.Χ. "http://www/uom.gr"
Κίτρινο χρώμα σημαίνει αρχείο π.Χ. "Αρχείο.txt"
Πράσινο χρώμα σημαίνει φάκελος π.Χ. "Φάκελος"
Μπλέ χρώμα σημαίνει κώδικας και μεταβλητές π.Χ "printf("Elements");"
Οι άλλες παρατηρήσεις συμβολίζονται με μαύρο χρώμα π.Χ "Settings"
Το νούμερο ως δείκτης με την παύλα προς τα αριστερά σημαίνει ότι η/οι πρόταση/εις που είναι υπογραμμισμένες σημειώνονται με footer και θα εξηγηθούν παρακάτω με τον αριθμό. π.Χ. Είναι η εργασία μου$_{1}$. Στο footer "1 = Εργασία 1 δείτε παρακάτω". Μετά από κάποες σειρές/σελίδες εμφανίζεται η περιγραφη του footer. Επίσης η σήμανση π.Χ. ". $_{2}$." σημαίνει η συντομογραφία της υπογραμμισμένης πρότασης με δείκτη το 2. Το 2 πάλι εμφανίζεται στο footer και μας εξηγεί την σημασία του.

Αρχικοποίηση Α' Υποερώτημα

Κατασκευή της βάσης της εργασίας

Άνοιξα το Visual Studio 2015 και πάτησα "file -> new Project -> Python Application ->" το έδωσα όνομα "Στέφανος_Κάππας_Εργασία_5" και πάτησα OK. Σε αυτό το project έκανα "add -> new item -> empty file.py ->" το έδωσα όνομα "Στέφανος_Κάππας_Εργασία_5.py" και εκεί θα γράψω το πρόγραμμα. Επίσης στο "Στέφανος_Κάππας_Εργασία_5" πρόσθεσα και ένα .pdf αρχείο. Το αρχείο το κατέβασα από το "https://openeclass.uom.gr" το οποίο λέει την εκφώνηση της εργασίας και το ονόμασα "Εκτσιριδοποίηση.pdf". Ο όρος εκτσιριροποίηση προέρχεται από το "εκ" + "τσιρίδα" επειδή ο όρος εκφώνηση προέρχεται από το "εκ" + "φωνάζω" που εάν φωνάζω όσο μπορώ τότε το κάνω τσιρίδα. Έτσι για αστείο αλλά και επίσης επειδή όταν χαίρονται οι βλάκες τσιρίζουν τόσο πολύ μέχρι να καταστρέψουν την φωνή τους.

Δημιουργία της λίστας προς εξέταση

Στο "Στέφανος_Κάππας_Εργασία_5.py" δημιούργησα μία δομή, η οποία επιστρέφει μία λίστα με 2 διαστάσεις (i χρήστες και j βιβλία). Αυτή την λίστα την ονόμασα "number_grid". Σε αυτή την λίστα πρόσθεσα στον κώδικα 3 χρήστες με 3 βιβλία και όταν εκτελείται το πρόγραμμα στην κονσόλα εμφανίζεται η λίστα με τους 3 χρήστες και τα 3 βιβλία (στατική λίστα)[1]. Αφού κατάφερα να κάνω αυτό το πράγμα έφτιαξα και κάποιες μεθόδους (οι οποίες θα αναφερθούν σε άλλη παράγραφο) που επεξεργάζονται διαβάζοντας την λίστα. Μετά αποφάσισα ότι η λίστα πρέπει να είναι δυναμική (το λέει η εκτσιριδοποίηση) παρά στατική και έτσι πρόσθεσα μεθόδους που επεξεργάζονται γράφοντας στην λίστα. Εκεί αντιμετώπισα πολλά προβλήματα αλλά στο τέλος το κατάφερα[2]. Στο τέλος την δυναμική λίστα την έκανα κενη μέσω κώδικα. Δηλαδή κράτησα μόνο την εντολή "['Χρήστης hid \ Βιβλίο hid']" που εμφανίζει στην οθόνη την πρώτη γραμμή και στήλη. Στο τέλος τροποποίησα τις μεθόδους που γράφουν στην λίστα με σκοπό να μην σβηστεί η πρώτη γραμμή και η πρώτη στήλη επειδή θα ήταν ολέθριο για το πρόγραμμα.

Υπενθυμίζω ότι η πρώτη γραμμή και πρώτη στήλη (i = 0, j = 0) εμφανίζεται το "Χρήστης hid \ Βιβλίο hid". Το j αφορά τον αριθμό των βιβλίων ενώ το i αφορά το πλήθος των χρηστών[3]. Το "id" το ονόμασα "hid". Το "id" προφέρεται "άιντι" ενώ το "hid" προφέρεται "Χάιντι" που ενοείται ότι είναι το κορίτσι των άλπεων και για αυτό το έκανα. Η εικόνα της Χάιντι είναι στο παράρτημα με όνομα Εικόνα 1.

Μέθοδοι που γράφουν στην λίστα

Στο "Στέφανος_Κάππας_Εργασία_5.py" πρόσθεσα 5 μεθόδους που γράφουν στην λίστα. Αυτές είναι οι "add_row(i)", "add_col(i, j)", "remove_row(i)", "remove_col(i)" και "select(i, j)". Μερικά είναι γραμμένα 2 φορές. Την μία φορά είναι στην ελληνική γλώσσα και την άλλη στην αραβική.

Η μέθοδος "add_row(i)" ζητάει πόσες γραμμές θέλει να προσθέσει ο διαχειρηστής. Αφού ο διαχειρηστής επιλέξει έναν αριθμό τότε το πρόγραμμα προσθέτει κάτω από την πρώτη γραμμή (i = 0) όσες γραμμές επιλέξει ο διαχειρηστής. Επίσης στην πρώτη στήλη (j=0) εμφανίζει την λέξη "χρήστης" και έναν αριθμό σύμφωνα με την i-ιοστή θέση. Π.χ. "Χρήστης 3". Αυτή η μέθοδος δέν έχει περιορισμούς αλλά θα αναφέρω τα προβλήματα σε άλλη ενότητα[2].

Η μέθοδος "add_col(i, j)", στην αρχή (ονομαζόταν add_2(i,j))[4] πρόσθετε μία στήλη και εκεί μας ρωτούσε "print("Βάλε βαθμολογία στην στήλη", j, "και γραμμή", i)" και ο διαχειρηστής επέλεγε κάποιο ακέραιο και έτσι το αποθήκευε στην λίστα. Ωστόσο

1 = βλέπε πίνακα 1
2 = βλέπε στην ενότητα "Προβλήματα που αντιμετώισα στις παραπάνω μεθόδους"

αυτή η πρακτική ήταν χρονοβόρα και προβληματική επειδή όταν ο διαχειρηστής ήθελε να προσθέσει πολλά βιβλία θα χρειαζόταν να την καλέσει ξανά και ξανά. Τα αποτελέσματα που θα προκύψουν είναι πρώτον να κάνει κάποιο λάθος (χαρακτήρα ή ακέραιο αντί για ρητό και εκτός τον ορίων από 0.0 εώς 10.0) και δεύτερον να χάσει πολύ χρόνο[2]. [4]Έτσι η μέθοδος τροποποιήθηκε στο να ζητάει πόσες στήλες θέλει να προσθέσεις ο διαχειρηστής και το πρόγραμμα προσθέτει όσες στήλες ζητήσει ο διαχειριστής. Η μόνη διαφορά είναι ότι προσθέτει τυχαίους αριθμούς απο το 0.0 εώς το 10.0 και όχι ότι θέλει ο διαχειριστής (ταχύτητα και χωρίς λάθη διαχειριστή). Επίσης στην πρώτη γραμμή (i=0) εμφανίζει την λέξη "βιβλίο" και έναν αριθμό σύμφωνα με την j-ιοστή θέση. Π.χ. "Βιβλίο 3". Υπενθυμίζω ότι τόσο η νέα έκδοση της μεθόδου έχει έναν περιορισμό. Ο περιορισμός είναι σε περίπτωση που δεν υπάρχουν χρήστες η μέθοδος δεν προσθέτει κανένα βιβλίο.

Η μέθοδος "remove_row(i)" αφαιρεί από την λίστα όσες γραμμές ζητήσει ο διαχειριστής. [2]. Η μέθοδος αυτή έχει έναν περιορισμό, που εμποδίζει να σβήσει όλες τις γραμμές επειδή εάν σβηστούν όλες τότε η προσθήκη γραμμών ή στηλών θα δημιουργούσε άλλα προβλήματα με την λίστα. Ποιο συγκεκριμένα είναι η απουσία της 1ης γραμμής που εμφανίζει το πλήθος των βιβλίων.

Η μέθοδος "remove_col(i)" αφαιρεί από την λίστα όσες στήλες ζητήσει ο διαχειριστής.[2]. Επίσης ο περιορισμός που υπάρχει είναι ίδιος με την "remove_row(i)" επειδή αν σβηστούν όλες οι στήλες τότε δεν θα εμφανίσει το πλήθος των χρηστών στη λίστα.

Η μέθοδος "select(i, j)" κάνει επιλογή κάποιων γραμμών και στηλών και τις μηδενίζει. Αυτή ζητάει από τον διαχειριστή από ποιά γραμμή μέχρι ποιά γραμμή και από ποιά στήλη μέχρι ποιά στήλη θέλει να μηδενίσει την λίστα. [2]. Ο περιορισμός είναι απλά το μέγεθος της λίστας. Αυτή η μέθοδος μπορεί να τροποποιηθεί στα επόμενα υποερωτήματα εάν ζητηθεί.

Μέθοδοι που διαβάζουν την λίστα

Στο "Στέφανος_Κάππας_Εργασία_5.py" πρόσθεσα 3 μεθόδους που διαβάζουν την λίστα. Αυτές είναι οι "emfanisi(i, j)", "user(i, j)", "book(i,j)".

Η μέθοδος "emfanisi(i, j)" εμφανίζει στην οθόνη τις γραμμές και στήλες της λίστας και το πλήθος των γραμμών και των στηλών της λίστας. Αυτή η μέθοδος στην πραγματικότητα δεν υπάρχει. [2]. Αυτή όμως είναι κωδικοποιημένη κάτω από την εντολή υπό συνθήκη "if list == 'Εμφάνιση':".

Η μέδοδος "user(i, j)" διαβάζει όλη την λίστα και συγκρίνει κατά γραμμή (χρήστη) τί βαθμολογίες έβαλε ο χρήστης σε κάθε βιβλίο και το πρόγραμμα επιστρέφει στην οθόνη το μέσο όρο βαθμολογίας στην οθόνη κατά χρήστη. Δηλαδή λέει "Ο χρήστης 2 βαθμολογεί με μέσο όρο 2,3 τα βιβλία". Αυτή η μέθοδος είναι γραμμένη τόσο στα ελληνικά όσο και στα αραβικά. Επίσης η μέθοδος αυτή είναι σχεδιασμένη να προσθέσει βαθμό στο κελί της λίστας που δεν υπάρχει βαθμολογία[2]. Αυτή η εξάσκηση θα βοηθήσει πολύ στο επόμενο υποερώτημα της εκτσιριδοποίησης.

Η μέθοδος "book(i,j)" κάνει την ίδια δουλειά με την "user(i, j)" με την διαφορά ότι εμφανίζει στην οθόνη τον μέσο όρο των βαθμολογιών ανά βιβλίο. Δηλαδή λέει "Το βιβλίο 2 έχει βαθμολογηθεί με μέσο όρο 2,3 από τους χρήστες".

Προβλήματα που αντιμετώπισα με τις παραπάνω μεθόδους το 2021

Στην εκτέλεση του προγράμματος αντιμετώπισα και πολλά (από τα οποία τα περισσότερα) επιλύσιμα προβλήματα. Στην δημιουργία της λίστας τα προβλήματα ήταν πολλά και διάφορα τόσο στις μεθόδους που διαβάζουν όσο σε αυτές που γράφουν στην λίστα. Παρατήρηση: στο τέλος των υπογραμμισμένων προτάσεων με

3 = βλέπε πίνακα 2
4 = Βλέπε Κώδικα 1

κατάληξη 2 είναι όλες οι προτάσεις που αναφέρουν πρόβλημα.

Στις μεθόδους “add_row(i)”, “add_col(i, j)” τα προβλήματα που αντιμετώπισα ήταν η μή εγγραφή του βαθμού στην λίστα, η εγγραφή περισσότερων βαθμών από το πλήθος των γραμμών ή στηλών και η βαθμολογία που ήταν ως string και όχι int ή float. Αποκλειστικά στην μέθοδο “add_row(i)” το πρόβλημα που αντιμετώπισα ήταν όταν προσθέτεις μία γραμμή το πρόγραμμα δεν πρόσθετε τον ίδιο αριθμο των στηλών στην λίστα αλλά πολύ μεγαλύτερο. Δηλαδή εάν πριν την προσθήκη γραμμής η λίστα έχει για παράδειγμα 2 γραμμές και 3 στήλες, Μετά την προσθήκη η λίστα θα έχει 3 γραμμές και 20 στήλες. Οι 20 στήλες προκαλούνται από το συνολικό πλήθος των χαρακτήρων στην τελευταία γραμμή (δηλαδή εάν η τρίτη γραμμή έχει την πληροφορία “['Χρήστης 3’, 1, 2, 3’]”, το πλήθος των χαρακτήρων είναι 20 χωρίς τις αγκύλες). Την λύση των προβλημάτων αυτών την έκανα με μεθοδική προσέγγιση στον κώδικα και πολλά πειράματα για να φτάσω στην τελική μορφή.

Στις μεθόδους “remove_row(i)”, “remove_col(i)” το πρόβλημα που αντιμετώπισα ήταν η διαγραφή γραμμής αντί για στήλης και το αντίστροφο. Επίσης άλλο πρόβλημα που αντιμετώπισα είναι η πλήρης διαγραφή της λίστας και η λάθως εμφάνιση του μέγεθους της λίστας στην μέθοδο “emfanisi(i, j)” μετά από την εφαρμογή τους.

Η μέθοδος “emfanisi(i, j)” είχε το πρόβλημα να μην εμφανίζει την αλλαγή της λίστας μετά την τροποποίηση της όταν ήταν αυτόνομη συνάρτηση. Το πρόβλημα επιλύθηκε με την τοποθέτηση της κάτω από την εντολή “if list == 'Εμφάνιση':” και έτσι σβήστηκε και το όνομα “emfanisi(i, j)”. Επίσης όταν η λίστα είχε 1 στήλη η εκτέλεσή της εμφάνιζε 3 γραμμές. Η λύση αυτού του προβλήματος ήταν με την προσθήκη μίας εντολής στο τέλος της συνάρτηστης “if i == 0: j = j - 2”. Επίσης προστέθηκε και η εντολή print που εμφανίζει το μέγεθος της λίστας για ασφάλεια.

Στην μέθοδο “select(i, j)”, το πρόβλημα που αντιμετώπισα στην αρχή ήταν ότι δεν αποθήκευε την αλλαγή του κελιού στην λίστα μετά την εκτέλεση της εντολής. Αυτή αντιμετωπίστικε με μία ποιο απλή συνάρτηση και με επαναληπτική δομή για εξοικονόμηση χρόνου.

Στις μεθόδους “user(i, j)”, “book(i,j)” τα προβλήματα ήταν διάφορα αλλά κοινά. Το ποιό σημαντικό πρόβλημα ήταν ότι μετά την προσθήκη γραμμής ή στήλης δεν την υπολόγιζε το πρόγραμμα ή ακόμα χειρότερα εμφάνιζε σφάλμα εκτέλεσης. Ένα άλλο πρόβλημα που αντιμετώπισα ήταν στην μετατροπή των τύπων δεδομένων (από ακέραιο σε συμβολοσειρά ή ρητό). Επίσης αντιμετώπισα το πρόβλημα που μετατρέπει ένα κενό κελί της λίστας σε κάποιο ρητό αριθμό (π.Χ. το 0.0) για την εύρηθμη λειτουργία της άσκησης.

Τα περισσότερα προβλήματα που ανέφερα μέχρι εδώ τα επίλυσα. Εξαίρεση είναι όταν προσθέτεις κάποιους χρήστες και κάποια βιβλία στην λίστα τότε δεν μπορείς μετά να προσθέσει και άλλους χρήστες παρά μόνο βιβλία επειδή εάν προσθέσεις και άλλους χρήστες τότε θα εμφανιστεί η λίστα με κενά κελιά με αποτέλεσμα σφάλματα εκτέλεσης στις συναρτήσεις που διαβάζουν την λίστα. Η μόνη λύση είναι η πλήρης διαγραγή της λίστας και από την αρχή η ξαναδημιουργία της. Επίσης στην συνάρτηση “user(i, j)” δεν μπορώ μέχρις στιγμής να τροποποιήσω τα κελιά που θέλω όταν εφαρμόζω την εντολή υπό συνθήκη. Στο τέλος ένα μικρό άλυτο πρόβλημα αντιμετώπισα το οποίο αφορά στην γλώσσα των μεθόδων “user(i, j)” και “book(i,j)”. Η λύση ήταν η διπλοεγγραφή τους δηλαδή στην πραγματικότητα υπάρχουν τα “user_greek(i, j)” και “user_arabic(i, j)”, “book_greek(i, j)” και “book_arabic(i, j)”.

Προβλήματα που αντιμετώπισα μετέπειτα δηλαδή το 2022

Μετά από ενάμισι χρόνο όταν τέσταρα τον κώδικα εντόπισα και επίλυσα διάφορα λάθη στον κώδικα και σφάλματα εκτέλεσης για την βέλτιστη λειτουργία του

προγράμματος. Ποιο συγκεκριμένα δεν θα αναφέρω κάθε λεπτομέρεια, αλλα θα αναφέρω ότι τροποποίησα τις εντολές με αυτό τον τρόπο.

Οι μέθοδοι στο πρόγραμμα πριν τις τροποποιήσω ήταν: “add_row(i)”, “add_col(i, j)”, “select_greek(i, j)”, “select_arabic(i,j)”, “remove_row(i)”, “remove_col(i)”, “user_greek(i, j)”, “user_arabic(i, j)”, “book_greek(i, j)”, “book_arabic(i, j)”. Αυτές τροποποιήθηκαν ως εξής.

Στην μέθοδο “add_row(i)” η λειτουργία της δεν άλλαξε καθόλου αλλά, η μεταβλητή ”a” μετατράπηκε σε “name” για να ξέρει ο αναγνώστης του κώδικα τι κάνουν οι μεταβλητές με βάση το πραγματικό τους όνομα.
Στην μέθοδο “add_col(i, j)” η λειτουργία της δεν άλλαξε καθόλου αλλά, οι μεταβλητές ”aa“ και “cc” μετονομάστηκαν σε “book” και “name” για τον ίδιο ακριβώς λόγο.

Στις μεθόδους “select_greek(i, j)”, “select_arabic(i,j)” πρώτα από όλα μετονόμασα τις μεταβλητές: “x”, “y”, “i”, “j”, “z”, “I”, “J” σε “x_start”, “y_start”, “x_finish”, “y_finish”, “value”, “ROW”, “COL” αντίστοιχα. Μετά πρόσθεσα την εντολή “if (x_start > 0 and x_finish >= x_start and x_finish <= i and y_start > 0 and y_finish >= y_start and y_finish <= j):” για να ελέγει το πρόγραμμα εάν τα κελιά που επιλέγει ο διαχειριστής υπάρχουν στην λίστα. Εάν δεν υπάρχουν τότε εμφανίζεται μύνημα στην οθόνη ότι επιλέγει εκτός ορίων κελιά. Δηλαδή εάν η λίστα έχει 3 γραμμές και 3 στήλες για απλότητα και ο διαχειριστής επιλέξει “x_start”, “x_finish”, : από 0 εώς 4 (όλα εκτός ορίων), από 1 εώς 4 (το “x_finish” είναι εκτός ορίων), από 4 εώς 3 (το “x_finish” είναι μικρότερο του “x_start”, από 2 εώς 1 εμφανίζει μύνημα λάθους. Παρατήρησα επίσης ότι το “x_start” δεν έχει όριο προς τα επάνω. Για να εκτελεστεί όμως η μέθοδος πρέπει να προστεθεί και το “x_finish”. Αν το “x_finish” είναι μεγαλύτερο του 3 τότε πάλι το πρόγραμμα εμφανίζει μύνημα λάθους. Τέλος ομαδοποίησα τις 2 μεθόδους “select_greek(i, j)”, “select_arabic(i,j)” σε μία, την “select(i, j, language)”, όπου η “language” καθορίζει σε ποιά γλώσσα θα εμφανίζονται τα μυνήματα. Εάν “language == ‘1’” τότε εμφανίζει ελληνικά. Εάν “language == ‘2’” τότε εμφανίζει αραβικά.

Στην μέθοδο “remove_row(i)” μετονόμασα την μεταβλητή από “b” σε “number_of_rows”. Επίσης πρόσθεσα και μία εντολή “else:” για την εμφάνιση του μυνήματος ότι δεν μπορεί ο διαχειριστής να διαγράψει περισσότερες γραμμές από το σύνολο των γραμμών της λίστας. Τέλος πρόσθεσα στην αναφορά της μεθόδου το “language” για την εμφάνιση του μυνήματος στις 2 γλώσσες και η μέθοδος ονομάζεται “remove_row(i, language)”

Στην μέθοδο “remove_row(i)” τα “b” και “a” τα ονόμασα σε “number_of_cols” και “row” αντίστοιχα. Επίσης τα ίδια έκανα και με την προηγούμενη μέθοδο και αυτή ονομάζεται “remove_col(i, language)”.

Στην “user_arabic(i, j)” πρώτα έλυσα το πρόβλημα της εμφάνισης του μέσου όρου των βαθμολογιών ανά χρήστη. Δηλαδή αντί για “d” ήταν “c”. Έπειτα και στην ελληνική έκδοση μετονόμασα τις μεταβλητές “a”, “b”, “c”, “d” σε “index”, “sum”, “value”, “average”. Έπειτα τροποποίησα την μέθοδο να εμφανίζει τα μυνήματα σε 2 γλώσσες ανάλωγα με την τιμή της “language” Έτσι οι 2 παραπάνω μέθδοι συγχωνεύτηκαν και σε μία, την “user(i, j, language)”. Στην “book_greek(i, j)”, “book_arabic(i, j)” έκανα ακριβώς τα ίδια με τις προυγούμενες 2 μεθόδους και έτσι η μέθοδος αυτή ονομάζεται “book(i, j, language)”.

Τέλος Α’ ερωτήματος.

Συνεργαγικό Φιλτράρισμα Β' Ερώτημα

Αρχικοποίηση συνεργατικού φιλτραρίσματος

Πριν αρχίσω το συνεργατικό φιλτράρισμα θα πρέπει να τροποποήσω την μέθοδο "select(i, j)", η οποία θα παίξει καθοριστικό ρόλο σε αυτό το μέρος. Η αλλαγή που έκανα είναι η επιλογή της βαθμολογίας που θα βάλει ο διαχειριστής στο επιλεγμένο εύρος i χρηστών και j βιβλίων που επέλεξε. Αυτή η βαθμολογία εξυπακούγεται ότι είναι ένας ρητός αριθμός που κυμαίνεται από το 0,0 εώς το 10,0. Αποφάσισα για το συνεργατικό φιλτράρισμα ότι η απουσία βαθμολογίας είναι 0,0 και το εύρος των βαθμολογιών ξεκινάει από το 0,1 (χειρότερος βαθμός) εώς το 10,0 (καλύτερος βαθμός). Τα πειράματα που θα κάνω σε αυτή την ενότητα βασίζονται στο "10 - Συστήματα Συστάσεων.pdf" και διαφάνεια 55 με τον πίνακα βαθμολογιών ακριβώς όπως είναι με την διαφορά όπου απουσιάζει βαθμολογία σημειώνεται ως μηδέν (0). Επίσης θα δημιουργήσω μία νέα λίστα στην αρχή του κώδικα (για πειραματικούς μόνο λόγους) που ονομάζεται "number_grid2" και με αυτή την λίστα θα κάνω όλα τα πειράματα που χρειάζονται για το συνεργατικό φιλτράρισμα και όταν πετύχουν ο πίνακας "number_grid2" θα αχριστευτεί και όλες οι διαδικασίες θα γίνονται με τον κλασικό πίνακα δηλαδή τον "number_grid".

Μέθοδοι συνεργατικού φιλτραρίσματος

Οι μέθοδοι συνεργατικού φιλτραρίσματος που θα κάνω στον κώδικα είναι όσοι έχουν ανατεθεί από την "Εκτσιριδοποίηση.pdf". Ποιό συγκεκριμένα οι μέθοδοι είναι: Jaccard Similarity, Cosine Similarity (Κοζάνη ομοιότητα), Dice Similarity, Adjusted Cosine Similarity (Αναθεματισμένη Κοζάνη ομοιότητα).

Η μέθοδος "jaccard()" που δημιούργησα για την Jaccard Similarity εφαρμόζει μακροσκοπικά τον τύπο "jaccard = (|ra|☹|rb|) / (|ra|☺|rb|)". Ο κώδικας απλά ζητάει από τον διαχειριστή να επιλέξει κάποιους 2 χρήστες και το πρόγραμμα συγκρίνει αυτούς τους 2 χρήστες και βγάζει το αποτέλεσμα στο Jaccard Similarity. Ποιό συγκεκριμένα η μέθοδος αυτή χειρίζεται τον πίνακα "number_grid2", έχει 3 μετρητές (ο "za" μετράει το πλήθος των βιβλίων που βαθμολόγησε ο πρώτος επιλεγμένος χρήστης, ο "zb" κάνει το ίδιο με αλλά για το δεύτερο επιλεγμένο χρίστη, το "Λ" που μετράει πόσα βιβλία βαθμολογήθηκαν από κοινού των επιλεγμένων χρηστών). Μετά την επιλογή των χρηστών από τον διαχειριστή η μέθοδος διαβάζει την γραμμή στο "number_grid2"$_5$ που επέλεξε ο διαχειριστής για τον πρώτο επιλεγμένο χρήστη και αποθηκεύει το πλήθος των βιβλίων που βαθμολόγησε στο "za". Συγχρόνως για κάθε βιβλίο που έχει βαθμολογηθεί$_6$ από τον πρώτο επιλεγμένο χρήστη η μέθοδος κάνει διαβάζει ξανά την γραμμή "number_grid2"$_5$ που επέλεξε ο διαχειριστής για τον δεύτερο επιλεγμένο χρήστη. Η μέθοδος και εδώ κάνει το ίδιο με πριν με την διαφορά ότι αποθηκεύει στο "zb" το πλήθος των βαθμολογημένων βιβλίων του δεύτερου επιλεγμένου χρήστη. Επιπλέον συγκρίνει το κάθε βιβλίο εάν βαθμολογήθηκε από κοινού και αν ναι αποθηκεύει στο "Λ" το πλήθος των βιβλίων που βαθμολογήθηκαν από κοινού. Μετά το διάβασμα των επιλεγμένων χρηστών από το "number_grid2", η μέθοδος υπολογίζει την ένωση των βαθμολογιών "U = za + zb - Λ" και στο τέλος κάνει την πράξη "jaccard = Λ / U" και εμφανίζει το αποτέλεσμα στην οθόνη.

Η μέθοδος "kozani()" που δημιούργησα για την Cosine Similarity εφαρμόζει μακροσκοπικά τον τύπο "Cos = $\text{Sum}_i(ra_i*rb_i)$ / ($\text{Sum}_i(ra_i\text{^}2)\text{^}0{,}5$ * $\text{Sum}_i(rb_i\text{^}2)\text{^}0{,}5$)". Η μέθοδος ζητάει τα ίδια από τον διαχειριστή σε σχέση με την "jaccard()". Ποιό συγκεκριμένα η μέθοδος αυτή χειρίζεται τον πίνακα "number_grid2"$_5$ και έχει 3 μετρητές (ο "AA" αθροίζει το γινόμενο της βαθμολογίας ανά βιβλίο από τους 2 επιλεγμένους χρήστες, ο "BB" αθροίζει τις βαθμολογίες του πρώτου επιλεγμένου

χρήστη, ο "CC" κάνει το ίδιο με το "BB" αλλά για τις βαθμολογίες του δεύτερου επιλεγμένου χρήστη). Η μέθοδος αυτή διαβάζει στήλη προς στήλη την πρώτη και την δεύτερη επιλεγμένη γραμμή του "number_grid2"$_5$ για να κάνει πολλαπλασιασμό των βαθμολογιών των κάθε βιβλίων από τους 2 επιλεγμένους χρήστες και το αποτέλεσμα αυτό το αυξάνει στον "AA". Μετά διαβάζει την στήλη της πρώτη επιλεγμένης γραμμής και την υψώνει στο τετράγωνο και αθρίζει το αποτέλεσμα στο "BB". Ομοίως κάνει το ίδιο για την δεύτερη επιλεγμένη γραμμή αλλά στο "CC". Μετά την ανάγνωση του βιβλίου εμφανίζει στη οθόνη το αποτέλεσμα του Cosine Similarity με την εντολή: "cos = AA / (math.sqrt(BB) * math.sqrt(CC))".

Η μέθοδος "dice()" που δημιούργησα για την Dice Similarity εφαρμόζει μακροσκοπικά τον τύπο "Dice= (2*|ra☹rb|) / (|ra|+|rb|)". Η μέθοδος ζητάει τα ίδια, όπως και οι προυγούμενες. Αυτή η μέθοδος έχει τόσο με τους μετρητές, όσο και με το διάβασμα της λίστας ακριβώς την ίδια λειτουργία με την "jaccard()". Μετά το τέλος της ανάγνωσης της λίστας η διαφορά είναι ότι υπολογίζει τον Dice Similarity. Ποιο συγκεκριμένα ο η εντολή είναι "dice = 2*Λ / (za + zb)" και το αποτέλεσμα το εμφανίζει στην οθόνη.

Η μέθοδος "adjusted_kozani_greek()" που δημιούργησα για την Adjusted Cosine Similarity ¨η αλλιώς Αναθεματισμένη Κοζάνη Ομοιότητα εφαρμόζει μακροσκοπικά τον τύπο
"ACS= (Sumi(ra - rma)*(rb - rmb)) / ((sum(ra - rma)^2)^0,5*(sum(rb - rmb)^2)^0,5)".
Η μέθοδος ζητάει τα ίδια, όπως και οι προυγούμενες. Αυτή η μέθοδος χωρίζεται σε 3 στάδια:
1) υπολογισμός μέσης βαθμολογίας ανά χρήστη,
2) αφαίρεση των βαθμολογιών με τους μέσους όρους της κάθε γραμμής και αποθήκευση του αποτελέσματος σε μία νέα λίστα "number_grid3",
3) Κοζάνη ομοιότητα πάνω στο "number_grid3",
4) Εμφάνιση του κανονικοποιημένου πίνακα με την συγκατάθεση του διαχειριστή.
Στο στάδιο 1 η μέθοδος διαβάζει όλη την λίστα "number_grid2"$_5$ και υπολογίζει τον μέσο όρο των βαθμολογιών ανά χρήστη-γραμμή. Δηλαδή ο χρήστης τάδε έχει βαθμολογίσει τόσα βιβλία και η μέση βαθμολογία είναι το άθροισμα των βαθμολογιών με το πλήθος των βιβλίων που βαθμολόγησε και όχι το συνολικό πλήθος. Δηλαδή η κάθε γραμμή έχει διαφορετικό μέσο όρο βαθμολογιών (υπενθυμίζω ότι γίνεται για όλους τους χρήστες και όχι για τους επιλεγμένους).
Στο στάδιο 2 η μέθοδος ξαναδιαβάζει όλη την λίστα "number_grid2"$_5$ και για κάθε στήλη-βιβλίο που έχει βαθμολογηθεί αφαιρεί την βαθμολογία με τον μέσο όρο βαθμολογίας του εκάστωτε χρήστη. Αυτό έχει ως αποτέλεσμα ότι η τιμή 0 στην βαθμολογία να μην είναι χειρότερη από την χειρότερη βαθμολογία αλλά να είναι η ουδέτερη -> άρα καλό ως προς τα βιβλια. Στο τέλος δημιουργεί τον "number_grid3" και αποθηκεύει όλο το αποτέλεσμα εκεί.
Στο στάδιο 3 η μέθοδος κάνει ακριβώς ότι έκανε η "kozani()" με την διαφορά ότι διαβάζει τον "number_grid3".
Στο στάδιο 4 η μέθοδος ζητάει από τον διαχειριστή εάν θέλει να εμφανιστεί ή όχι το "number_grid3". Υπενθυμίζω ότι το "number_grid3" είναι ο κανονικοποιμένος πίνακας βαθμολογιών.

Προβλήματα και λύσεις που αντιμετώπισα το 2021

Εδώ αντιμετώπισα πολλά και διάφορα προβλήματα, τα οποία όλα ήταν επιλύσιμα. Επίσης το κόλπο που έκανα με τον πίνακα από το "10 - Συστήματα Συστάσεων.pdf" και διαφάνεια 55 (που το αποθήκευσα στο "number_grid2"$_7$) με βοήθησε πολύ. Επειδή το "number_grid2" χρησιμοποιεί φυσικούς αριθμούς και μάλιστα το εύρος

ήταν από το 0 εώς το 5 με αποτέλεσμα να καταλάβω καλύτερα τα εύρη παρά με το "number_grid" που χρησιμοποιεί ρητούς αριθμούς και πάρα πολλούς. Έτσι στην αρχή εφάρμοσα το "number_grid2"σε όλες τις μεθόδους συνεργατικού φιλτραρίσματος και όταν κατάφερα να επιλύσω όλα τα προβλήματα (που θα τα αναφέρω στην επόμενη πρόταση) τότε έβαλα το "number_grid".
Τα υπόλοιπα προβλήματα ήταν η λανθασμένη επιλογή της γραμμής-χρήστης της λίστας με αποτέλεσμα να μην συγκρίνω σωστούς χρήστες και ακόμα να συγκρίνω όλη την λίστα και να κρασάρει το πρόγραμμα (στην περίπτωση που διαβάζει την πρώτη στήλη επειδή αυτή εμφανίζει το όνομα του χρήστη και όχι την βαθμολογία). Εφόσον κατάφερα το παραπάνω πρόβλημα αντιμετώπισα το πρόβλημα της σύγκρισης των βαθμολογιών του ίδιου βιβλίου από 2 χρήστες. Η λύση ήταν η διπλή επαναληπτική δομή (η εξωτερική είναι του πρώτου επιλεγμένου χρήστη ενώ η εσωτερική είναι του δεύτερου επιλεγμένου χρήστη) αλλά και εκεί αντιμετώπισα το πρόβλημα της ένωσης. Η λύση ήταν πολλές μεταβλητές ως μετρητές και πολλές εμφολευμένες εντολές υπό συνθήκη.
Το Cosine Similarity ή αλλιώς η Κοζάνη Ομοιότητα ήταν αρκετά εύκολη υπόθεση. Παρόλο που σύγκρινε την βαθμολογία από τους 2 επιλεγμένους χρήσες του κάθε βιβλίου δεν χρειαζόταν τόσα εμφολευμένα if και for loops. Το μόνο επιπρόσθετο πρόβλημα ήταν η λανθασμένη άθροιση και πολλαπλασιασμώς των μετρητών (που επιλύθηκε γρήγορα) και η χρήστη των ριζών.
Το Dice Similarity παρόλο που είχε την δυσκολία του Jaccard ήταν εύκολο επειδή είχα καταλάβει πως επιλύνεται το παραπάνω πρόβλημα.
Το Adjusted Cosine Similarity ήταν το ποιο δύσκολο επειδή κάνει 3 κύριες λειτουργίες. Το μεγαλύτερο μέρος του προβλήματος ήταν στην κατασκευή της "number_grid3" χωρίς να την χαλάσω. Εκεί αντιμετώπισα προβλήματα με την πρώτη γραμμή και πρώτη στήλη επειδή όταν υπολογίζω την κανονικοποιημένη βαθμολογία πρώτον δεν εμφανίζει ποιός χρήστης βαθμολόγησς ποιό βιβλίο. Άλλο πρόβλημα ήταν ο υπολογισμός της κανονικοποιημένης βαθμολογίας επειδή έβαζε άλλαν ντάλων νούμερα και όχι σωστά. Το άλλο μέρος αντιμετωπίστικε ποιό εύκολα και μεθοδικά με πολλές μεταβλητές για να βγάλει το κατάλληλο αποτέλεσμα. Όσον αφορά την μετάβαση από την "number_grid2" στην "number_grid" κατάφερα να δείξω στο πρόγραμμα ποιές βαθμολογίες δεν θεωρούνται βαθμολογίες. Τα πειράματα που έκανα στην αρχή ήταν ο χωρισμός των βαθμολογιών κάτω του 5,0 και άνω του 5,0. Όταν το κατάφερα αποφάσισα να θέσω το 0,0 ως απουσία βαθμολογίας και το 0,1 ως ελάχιστη βαθμολογία και το 10,0 ως βέλτιστη βαθμολογία.
Είχα και άλλα προβλήματα που αντιμετώπισσα αλλά δεν είναι τόσο ουσιώδη και επίσης θα κάνω πολύ χρόνο να τα εξηγήσω. Το μόνο σίγουρο είναι ότι όλα επιλύθηκαν.

Προβλήματα που αντιμετώπισα μετέπειτα δηλαδή το 2022

Όπως είχα αναφέρει πριν, θα αναφέρω τις αλλαγές που έκανα στον κώδικα και όχι τα ενδιάμεσα στάδια. Οι μέθοδοι που εξέτασα ήταν: "jaccard_greek()", "jaccard_arabic()", "kozani_greek()", "kozani_arabic()", "dice_greek()", "dice_arabic()", "adjusted_kozani_greek()", "adjusted_kozani_arabic()".

Στις "jaccard_greek()", "jaccard_arabic()" αρχικά μετέφερα τις εντολές: "print("Ποιούς χρήστες θέλεις να συγκρίνεις;"); a = input(); b = input(); a = int(a); b = int(b)" στην "main()". Έπειτα, τις μεταβλητές "ja", "za", "jb", "zb", οι οποίες βρίσκονται στο σώμα του κώδικα τις μετονόμασα σε "j_user", "user_positive", "j_compare_user", "compare_user_positive" αντίστοιχα. Ομοίως και τις μεταβλητές που μετέφερα στην "main()" δηλαδή τις, "a", "b" τις μετονόμασα σε "user",

“compare_user” αντίστοιχα και επίσης αυτές τις τοποθέτησα στην αναφορά της μεθόδου. Ύστερα πρόσθεσα μία εντολή υπό συνθήκη την:
“if (user > 0 and user <= i and compare_user > 0 and compare_user <= i):” για να ελέγχει το πρόγραμμα εάν ο διαχειριστής που συγκρίνει τις βαθμολογίες του χρήστη με άλλον χρήστη εάν βρίσκονται εντός της λίστας και εάν δεν είναι τότε εμφανίζεται μύνημα σφάλματος. Στο τέλος με την προσθήκη της μεταβλητής “list” στην αναφορά της μεθόδου, η οποία είναι και αυτή στην “main()” που αυτή είναι η μεταβλητή είσόδου αλφαρηθμιτικού για την επιλογή των μεθόδων που θα ακολουθήσει ο διαχειριστής. Αυτή καθορίζει το τι θα εμφανίσει η μέθοδος στην οθόνη και την αποθήκευση του αποτελέσματος στην λίστα “Similarity_value” που παλιά ονομαζόταν “QQ” την τιμή του τρέχοντος αποτελέσματος για την περίπτωση αυτή την τιμή της “jaccard” και της τιμής του 2ου χρήστη που επιλέγει ο διαχειριστής για να συγκρίνει με συνεργατικό φιλτράρισμα με τον 1ο, δηλαδή την “compare_user” και αποθηκεύει την τιμή στην λίστα “Similarity_index” που παλιά αυτή ονομαζόταν “QQQQ”. Τέλος η “language” καθορίζει σε πιά γλώσσα θα εμφανίζονται τα μυνήματα στην οθόνη και η “prin” θα την αναφέρω σε άλλη ενότητα τί κάνει. Έτσι οι 2 μέθοδοι “jaccard_greek()” και “jaccard_arabic()” συγχωνεύτηκαν σε μία, την “jaccard(user, compare_user, language, list, prin):”.

Στις “kozani_greek()”, “kozani_arabic()” έκανα ακριβώς ότι έκανα και στην προυγούμενη μέθοδο. Η μόνη διαφορά είναι στην ονοματοδοσία των μεταβλητών. Πιο συγκεκριμένα οι μεταβλητές: “j”, “AA”, “BB”, “CC”, “bb”, “cc” μετονομάστηκαν σε “j_user”, “user_x_compareuser”, ”sum_user”, “sum_compareuser”, “value_user”, “value_compare_user” αντίστοιχα. Στο τέλος τις 2 παραπάνω μέθοδους τις συγχώνευσα σε μία, την
“kozani(user, compare_user, language, list, prin):”

Στις “dice_greek()”, “dice_arabic()” έκανα όακριβώς ότι έκανα και στην “jaccard(user, compare_user, language, list, prin):”, με την διαφορά ότι τις συγχώνευσα σε μία, την “dice(user, compare_user, language, list, prin):”.

Στις “adjusted_kozani_greek()”, “adjusted_kozani_arabic()” τροποποίησα τις μεταβλητές: “AA”, “BB”, “CC” σε “user_x_compareuser”, “user_sum”, “compareuser_sum” αντίστοιχα. Τις υπόλοιπες λειτουργίες τις έκανα όπως και στις 3 παραπάνω μεθόδους. Η μόνη διαφορά είναι ότι έσβησα τις εντολές “bbbb = number_grid[bbb][0]; QQQ.append(bbbb)” και την “QQQ = []” στην “main:” επειδή το “QQQ” δεν χρησιμεύει σε τίποτα. Απλά αυτή η λίστα εμφανίζει τα ονόματα των χρηστών που έχουν συγκριθεί. Δηλαδή εάν ο διαχειρηστής πατήσει ΑΚοζάνη -> user == 1 -> compare_user == 3 -> 1 -> 1” τότε στον “QQQ” θα αποθηκευτεί η τιμή “Χρήστης 3” που δεν χρησιμεύει σε τίποτα, επειδή οι χρήστες είναι αριθμημένοι κατά αύξουσα σειρά. Έτσι συγχωνεύτηκαν οι μέθοδοι σε μία, την “adjusted_kozani(user, compare_user, language, list, prin):”.

Τέλος Β’ ερωτήματος α’ υποερώτημα.

Πρόβλεψη Βαθμολογιών Β' Ερώτημα

Μέσος όρος των βαθμολογιών

Καταρχάς στο πρώτο υποερώτημα έκανα 2 μεθόδους υπολογισμού μέσης βαθμολογίας. Η μία είναι η βαθμολογία ανά βιβλίο "book(i,j)" και η άλλη είναι η βαθμολογία ανά χρήστη "user(i, j)". Παρόλο που στην αρχικοποίηση είπα ότι είναι οι παγιωμένες μεθόδους αυτό στην πραγματικότητα δεν ισχύει. Έκανα κάποιες αλλαγές για να λειτουργούν ακόμη πιο σωστά. Ποιό συγκεκριμένα όταν εμφανίζει στην λίστα ο βαθμός 0,0 η 2 παραπάνω μέθοδοι το θεωρούσαν στην Αρχικοποίηση 0,0 και όχι 5,0. Θυμηθείτε ότι στην αρχικοποίηση αυτές οι μέθοδοι όταν βρίσκουν κενό κελί (δηλαδή '') το μετατρέπουν σε 0,0. Αυτό όμως δεν ισχύει, επειδή πρώτα από όλα δεν επιτρέπεται να υπάρχει κενό κελί και δεύτερον το εύρος βαθμολογιών είναι από το 0,0 εώς το 10,0. Έτσι εγώ αποφάσισα να πώ ότι το 0,0 είναι η απουσία βαθμολογίας και το 0,1 η ελάχιστη βαθμολογία.

Στην μέθοδο "book(i,j)" οι αλλαγές ήταν ελάχιστες αλλά συγκεκριμένες. Λίγο πρίν την επαναληπτική δομή έθεσα το "c = 0" για αποφυγή επαανάγνωσης της ίδιας λίστας στην περίπτωση που αλλοιωθεί. Στην επαναληπτική δομή έβαλα το "c = row[e]" πρίν από τις πράξεις, επειδή δεν χρειάζεται μετατροπή τύπου από string σε int και άλλα και έτσι έσβησα την εντολή "c = int(row[e])". Πριν όμως από τον υπολογισμό του μέσου όρου βαθμολογίας (δηλαδή στην αρχή κάθε επανάληψης) υπήρχε η εντολή υπό συνθήκη "if row[e] is '':" και την έκανα "if c == 0.0:" επειδή δεν υπάρχει κενό κελί και η κενή βαθμολογία είναι 0,0. Αυτές ήταν οι αλλαγές μόνο 3. Ομοίως τις ίδιες (μάλλον παρόμοιες) αλλαγές έκανα στην μέθοδο "user(i, j)" που δεν χρειάζεται να τις πώ.

Πρόβλεψη βαθμού με k-κοντινότερων γειτόνων

Στην "adjusted_kozani_greek()" στο τέλος της μεθόδου πρόσθεσα κάποιες εντολές που δημιουργούν 3 λίστες (καθολικές). Η μία λίστα λέει τις τιμές της Αναθεματισμένης Κοζάνης Ομοιότητας, η άλλη λέει το όνομα του χρήστη, η τρίτη λέει την θέση του δεύτερου επιλεγμένου χρήστη. Αυτές τις 3 λίστες τις κάνει append. Εγώ δημιούργησα 2 μεθόδους: την "predict_1()" που κάνει την πρόβλεψη απλού μέσου όρου και την "predict_2()" που κάνει την πρόβλεψη σταθμισμένου μέσου όρου. Τόσο στην "predict_1()" όσο και στην "predict_2()" η διαδικασία είναι η ίδια. Δηλαδή, ρωτάει το πρόγραμμα στον διαχειριστή ποιό κελί θέλει να επιλέξει για να κάνει την πρόβλεψη βαθμολογίας (αρκεί αυτό το κελί να είναι 0.0). Μετά την σωστή επιλογή των κελειών της λίστας το πρόγραμμα ρωτάει στον διαχειριστή πόσο k κοντινότερους γείτονες θα επιλέξει (το εύρος είναι από το 1 εώς το 10). Αφού επιλέξει κάποιο k, το πρόγραμμα διαβάζει τις 3 λίστες που δημιουργήθηκαν από το "adjusted_kozani_greek()" και βρίσκει τα κελειά με τους k καλύτερους συντελεστές ομοιότητας. Αυτές τις πληροφορίες τις αποθηκεύει σε άλλες 3 (τοπικές) λίστες, οι οποίες θα χρησιμεύσουν στην πρόβλεψη βαθμολογίας.

Στην περίπτωση που επέλεξε το "predict_1()" με k = 1 εώς 10 το πρόγραμμα διαβάζει την στήλη (επιλεγμένο βιβλίο), το οποίο έχει βαθμολογηθεί από τους k κοντινότερους χρήστες και αθρίζει τις βαθμολογίες και τις διαιρεί με το k. Να υπενθυμίσω ότι στην πραγματικότητα δεν διαβάζει το "number_grid" στην επαναληπτική δομή, αλλά διαβάζει τις 3 τοπικές λίστες και με αυτές βγάζει την πρόβλεψη του βαθμού. Στο τέλος το αποτέλεσμα αποθηκεύεται στην επιλεγμένη θέση του "number_grid".

Στην περίπτωση που επέλεξε το "predict_2()" με k = 1 εώς 10 το πρόγραμμα κάνει ακριβώς το ίδιο με το "predict_1()" με την διαφορά ότι πολλαπλασιάζει τον κάθε

βαθμό με τον συντελεστή του “adjusted_kozani_greek()” του κάθε χρήστη.
Στην περίπτωση που επέλεξε το “predict_1()” για k εκτός των ορίων, το πρόγραμμα διαβάζει όλους τους χρήστες που βαθμολόγησαν το επιλεγμένο βιβλίο, τις αθρίζει και τις διαιρεί με το πλήθος των χρηστών και αποθηκεύει το αποτέλεσμα στην επιλεγμένη θέση του “number_grid”
Στην περίπτωση που επιλέξει κελί με βαθμολογία το πρόγραμμα δεν κάνει την πρόβλεψη και εμφανίζει ένα μύνημα.

Καθαρισμός
Στο τέλος πρόσθεσα και μία μέθοδο που στην πραγματικότητα δεν υπάρχει με όνομα, αλλά είναι στην κύρια συνάρτηση κάτω από την εντολή “if list == ‘Καθαρισμός’:”. Αυτή η μέθοδος σβήνει τις 3 (καθολικές) λίστες. Αυτή η μέθοδος αποτρέπει το πρόγραμμα να εμφανίσει σφάλματα κατά την επανεκτέλεση των προβλέψεων και έτσι πρέπει ο χρήστης να ξανά καλέσει την μεθοδο “adjusted_kozani_greek()” και μετά τις προβλέψεις. Δεν θα αναφέρω πρόβλημα.

Προβλήματα που αντιμετώπισα το 2022
Θα αναφέρω τί τροποποίησα και όχι τί προβλήματα αντιμετώπισα. Οι μέθοδοι που τροποποίησα ήταν: “predict_1_greek():”, “predict_2_greek():”, “predict_1_arabic():”, “predict_2_arabic():”.
Στις “predict_1_greek():”, “predict_1_arabic():”, δηλαδή η πρόβλεψη απλού μέσου όρου, που υπολογίζει τους κοντινότερους γείτονες με βάση τις τιμές της “Similarity_value” κατά φθίνουσα σειρά απλά κάνει διαίρεση του αθρίσματος των βαθμολογιών με το “k”. Βασικά τις έσβησα και τις ξαναέφτιαξα από την αρχή. Χωρίς να εξηγώ κάθε γραμμή και μεταβλητή του κώδικα. Οι διαφορές είναι ότι τις μεταβλητές “x”, “y” τις τοποθέτησα στην “main:”. Η “list” ελέγχει εάν ο διαχειριστής επιλέξει “Πρόβλεψη 2” η μέθοδος θα εμφανίζει το μύνημα για ποιά τιμή “k” θα επιλέξει. Εάν ο διαχειριστής επιλέξει “Γ” η μέθοδος δεν θα αφήσει τον διαχειριστή να επιλέξει κάποιο “k”. Η ”language” ξέρουμε τι κάνει. Η “prin” θα την αναφέρω αργότερα. Στην συνέχεια διαβάζει την λίστα “Similarity_value” μαζί με την λίστα “Similarity_index” και ταξινομεί τις λίστες σε φθίνουσα σειρά με βάση την “Similarity_value”. Στο τέλος αθροίζει τις τιμές του “number_grid[index][y]” όπου το “index” είναι η τρέχουσα γραμμή σχετικά με την τιμή της “Similarity_index[]”. Για παράδειγμα, έστω μία στήλη της “number_grid” είναι: “βιβλίο 3; 2; 0; 1; 4” και ένα συνεργατικό φιλτράρισμα που αποθηκεύει στις λίστες “Similarity_value”, “Similarity_index” τα αποτελέσματα “0.123, 0.432, 0.846”, “1, 3, 4” αντίστοιχα. Υπενθυμίζω ότι το συνεργατικό φιλτράρισμα δεν γίνεται με τον εαυτό του και για αυτό τον λόγο δεν εμφανίζεται το “2” στο “Similarity_index”. Η μέθοδος κάνει sort και τα αποτελέσματα είναι: “0.846, 0.432, 0.123”, “4, 3, 1”. Με βάση την ταξινομημένη λίστα η πρόβλεψη έστω για k = 3 κάνει 4 + 1 + 2 = 7 και το αποτέλεσμα το διαιρεί με το k. Άρα το αποτέλεσμα είναι 7/3. Το “g” μενοτομάστηκε σε “predict_value”, πρόσθεσα το “rest_of_k” που δεν υπήρχε πριν για να κάνει τις προβλέψεις και για “k < j-1” και τέλος πρόσθεσα μία νέα μεταβλητή, την “RANGE”, που είναι “k_limit - 1” . Επίσης πρόσθεσα στο τέλος με την “list” τι θα κάνει εάν είναι το “list == ‘Πρόβλεψη 2’ ή ‘Γ’” με την δεύτερη επιλογή να αναφερθεί σε επόμενη ενότητα. Έτσι αυτές οι 2 μέθοδοι συγχονεύτηκαν σε μία, την “predict_2(x, y, k, language, list, prin):”.
Στις “predict_2_greek():”, “predict_2_arabic():” έκανα τα ίδια όπως και στις προυγούμενες μεθόδου. Χωρίς να διευκρινίσω περισσότερο εάν ο διαχειριστής θέλει να εκτελεστεί αυτή η μέθοδος πρέπει να επιλέξει “list == ‘Πρόβλεψη 3’”. Οι

μεταβλητές “b”, “c”, ”d”, “e”, “f”, “g” μετονομάστηκαν σε , “index”, “similarity_value”, “product”, “sum_of_product”, “sum_of_similarity_value”, “predict_value”. Η μεταβλητή “a” καταργήθηκε επειδή δεν χρειάζεται. Έτσι αυτές οι 2 μέθοδοι συγχωνεύτηκαν σε μία, την “predict_3(x, y, k, language, list, prin)”.

Τέλος πρόσθεσα μία ακόμη μέθοδο, την “predict_1(x, y, k, language, list, prin)”, όπου αυτή μοιάζη με τις “predict_1_greek():”, “predict_1_arabic():”, μόνο που δεν χρησιμοποιεί τις λίστες “Similarity_index” και “Similarity_value”. Με λίγα λόγια, αφού ο διαχειριστής επιλέξει “list == ‘Πρόβλεψη 1’” και επιλέξει επίσης κάποιο κελί προς εξέταση (που πρέπει να είναι = 0.0, όπως και στις προυγούμενες μεθόδους) και κάποιο “k” και το πρόγραμμα κάνει πρόβλεψη με βάση τους επόμενους γείτονες. Με λίγα λόγια χρησιμοποιεί κάποιες μεταβλητές, τις “search_row” που αυξάνεται κατά 1 κάθε φορά που αθροίζει την τιμή του επόμενου γείτονα “row[y]” της “number_grid” εάν αυτή είναι > 0.0. Η “rest_of_k” ελέγχει σύμφωνα με τον διαχειριστή πόσους επόμενους γείτονες θα εξετάσει η μέθοδος. Επίσης αυτή ελέγχεται με την προυγούεμενη μεταβλητή ως “if search_row > x and rest_of_k > 0:” για τους επόμενους γείτονες μέχρι το “row == j”. Εάν ξεπεράσει το όριο τότε ελέγχει τον πίνακα από την αρχή με την εντολή “if search_row > 0 and search_row < x and rest_of_k > 0:”*. Τέλος το ”sum” αθροίζει τις τιμές της “row[y]” της “number_grid” και το “predict_value” διαιρεί το “sum” με το “k”. * = Έστω ότι ο πίνακας “number_grid” εχει μία στήλη = “’Βιβλίο 3’; 3; 0; 2” και “k = 2”. Η μέθοδος θα κάνει πρόβλεψη για “x = 2”, “y = 3” και πρώτα αθροίζει το 2 όπου το “row[y]” είναι στο “search_row = 3 = y + 1 = j”. Την επόμενη φορά αθροίζει στο “sum” το 3 όπου το “row[y]” είναι στο “search_row = 1 = y + 1 - j” δηλαδή εάν φτάσεις στο τέλος ξεκίνα από την αρχή. Στο τέλος διαιρεί το 3 + 5 με το “k = 2”. Άρα το αποτέλεσμα είναι 4.

Εξοικονόμηση κώδικα το έκανα και αυτό το 2022

Μία καινοτόμα λειτουργία που έκανα είναι η προσθήκη μίας ακόμη μεθόδου, την “compare(list, language):”. Στην πραγματικότητα, όταν ο διαχειριστής επιλέγει να εκτελεστεί μια μέθοδος όλου του Β’ ερωτήματος, στην πραγματικότηα αυτή δεν εκτελείται στην “main:”, αλλά στην “compare(list, language):”. Ο λόγος που το έκανα είναι επειδή πάντα σε όλες αυτές τις μέθόδους που έκανα, το πρόγραμμα ζητάει από τον διαχειριστή 2 μεταβλητές να επιλέξει, τις “a”, “b”. Η επιλογή όμως της μεθόδου από την “main:” που θα επιλέξει ο διαχειριστής είναι μόνο μέσω της “list”. Τέλος η μέθοδος “Καθαρισμός” Καθαρίζει και την “number_grid2”. Ο λόγος που το έκανα είναι πρώτον ότι η “main:” γίνεται πιο μικρή και κατανοητή και δεύτερον, απαλείφονται τα επαναλαμβανόμενα μυνήματα στην οθόνη και οι μεταβλητές εισαγωγής από τον διαχειριστή για να εκτελεστούν αυτές οι μέθοδοι.

Τέλος Β’ ερωτήματος β’ υποερώτημα.

Πρόβλεψη Βαθμολογιών Ολοκληρωμένο Γ' Ερώτημα

Αρχικοποίηση Ολοκληρωμένου Συστήματος Προβλέψεων Βαθμολογιών

Είχα κάνει και πέρισυ ένα σύστημα που αρχικά αντιγράφει το "number_grid" σε μία νέα λίστα. Μετά κάνει μηδενισμός κάποιων κελιών του "number_grid". Έπειτα κάνει συνεργατικό φιλτράρισμα μεταξύα χρηστών από τις υπάρχουσες μεθόδους που υλοποίησα παραπάνω. Ύστερα κάνει πρόβλεψη βαθμολογιών (αυτές που υλοποίησα παραπάνω) για τα κενά κελιά και στο τέλος εμφανίζει τις 2 λίστες για σύγκριση παλιάς με νέας.

Εκεί αντιμετώπισα διάφορα προβλήματα. Τα κυριότερα ήταν: α) ότι αντιγράφει το τελικό αποτέλεσμα του "number_grid" στην νέα λίστα. β) Ο μηδενισμώς των κελιών του "number_grid" ήταν χωρίς τυχαιοποίηση συχνωτήτων ή θέσεων για μηδένισμα. Για παράδειγμα έστω ο κώδικας:
"for row in range (1, i): for col in range (1, j): if row + col % 2 and row + col % 3: number_grid[row][col] = 0.0;", δεν μηδενίζει τυχαία σημεία αλλά τα ίδια σχετικά με το μέγεθος του πίνακα και με την ίδια συχνότητα. γ) Το συνεργατικό φιλτράρισμα που κάνει είναι πρώτα από όλα με το χέρι, δηλαδή ο διαχειρηστής για κάθε πρόβλεψη που κάνει το σύστημα πρέπει να επιλέξει ξανά και ξανά ποια μέθοδο ομοιότητας χρηστών. Το μόνο θετικό είναι ότι κάνει για έναν προς όλους αυτόματα, αλλά πάλι οδηγεί σε ένα ακόμα πρόβλημα. δ) Η ομοιότητα μεταξύ του ίδιου χρήστη που πάντα βγάζει αποτέλεσμα == 1, που δεν το θέλουμε. ε) Οι προβλέψεις βαθμολογιών είχαν κάποια προβλήματα σχετικά με τα αποτελέσματα που βγάζουν και σφάλματα εκτέλεσης. ζ) Δεν τα επίλυσα τα προβλήματα και δεν κατάφερα να κάνω την σύγκριση των μεθόδων και όλης γενικά της άσκησης.

Επανασχεδιασμός Ολοκληρωμένου Συστήματος Προβλέψεων Βαθμολογιών

Δημιούργησα μία μέθοδο με όνομα "gamma(language, count):" όπου το "language" ξέρουμε τί κάνει. Το "count" απλά μετράει πόσες φορές εκτελείται αυτή η μέθοδος. Αυτή κάνει μία αρχικοποίηση με 2 λίστες, τις "i_grid", "j_grid" που απλά αποθηκεύουν τις θέσεις των μηδενικών = άγνωστων βαθμολογιών (ποιά γραμμή και στήλη αντίστοιχα βρίσκονται) του "number_grid". Επίσης αρχικοποιείται και με μία ακόμη μεταβλητή, την "number_of_zeros = 1", που μετράει πόσα μηδενικά έχει το "number_grid" μετά τον μηδενισμός της λίστας.

Στην συνέχεια κάνει τον έλεγχο "if count == 0", για να αποφασίσει εάν θα αντιγράψει το "number_grid" στο νέο "number_grid2". χωρίς να αναφέρω τις λεπτομέριες για την λειτουργία αυτή, αυτή μοιάζει με την λειτουργία της κατασκευής του "number_grid3" στην "adjusted_kozani(user, compare_user, language, list, prin):" και λίγο στο αρχικό της μέρος σαν την "add_row(i):".

Μετά διαβάζει το "number_grid" κατά κελί, δηλαδή με τις εντολές: "for row in range(1, i+1): for col in range(1, j+1):". Ελέγχει εάν το τρέχον κελί είναι κενό, δηλαδή εάν "number_grid[row][col] == 0.0". Εάν ισχύει, τότε απλά αποθηκεύει την τρέχουσα τιμή της "row" στην λίστα "i_grid" μέσω της εντολής "i_grid.append(row)". Το ίδιο ακριβώς γίνεται και για την αποθήκευση της τρέχουσας τιμής της "col" στν λίστα "j_grid" και επίσης αυάνει το "number_of_zeros" κατά 1. Εάν όμως το τρέχων κελί της "number_grid" δεν είναι κενό τότε με την μεταβλητή "new" που βγάζει τυχαίους αριθμούς από το 0 εώς 9 (επειδή θέλω εγώ το πλήθος των άγνωστων μεταβλητών να είναι περίπου 10%) και μετά κάνει έλεγχο εάν το "new == 0", και εάν είναι τότε μηδενίζει το τρέχον κελί της "number_grid". Επίσης κάνει τις αλλαγές στις μεταβλητές "i_grid", "j_grid", "number_of_zeros" όπως έκανε και όταν το τρέχον κελί ήταν εξαρχής κενό.

Αφού έγιναν αυτές οι παραπάνω λειτουργίες. Το πρόγραμμα ελέγχει εάν υπάρχουν κενά κελιά στην "number_list" με την εντολή "if number_of_zeros > 1" και εάν δεν υπάρχουν τότε σταματάει η μέθοδος. Εάν υπάρχει έστω και ένα, δηλαδή τότε με βάση ποιά γλώσσα επέλεξε εξαρχής ο διαχειριστής εμφανίζονται τα μηνύματα, με ποιά μέθοδο ομοιότητας θα ακολουθήσει για όλα τα κενά κελιά, ποιά πρόβλεψη θα επιλέξει, πόσους k-γείτονες θα επιλέξει (υπενθυμίζω ότι είναι fix για κάθε κενό κελί και όχι για καθένα χωριστά) και επίσης το πρόγραμμα ελέγχει εάν το "k" είναι εντός τον ορίων (από 1 εώς i-1) και εάν δεν είναι τότε μύνημα σφάλματος και σταματάει η εκτέλεση της μεθόδου. Στο τέλος χρησιμοποιεί και την πολυσυζητημένη μεταβλητή, την "prin" που ρωτάει εάν θα εμφανίζονται τα αποτελέσματα ή όχι. Εάν "prin == 1" τότε τα συνεργατικά φιλτραρίσματα θα εμφανίζουν τα αποτελέσματα στην οθόνη παρόλο που το "list == Γ" και όχι το όνομα της μεθόδου. Ομοίως γίνεται και για την εμφάνιση των αποτελεσμάτων των προβλέψεων για κάθε κενό κελί. Επίσης εμφανίζει και τις θέσεις των κενών κελιών του "number_grid" με τα "i_grid" και "j_grid" και την εμφάνιση των αλλαγών στον "number_grid" μετά από κάθε πρόβλεψη. Εάν "prin" δεν είναι "1", απλά κάνει τις πράξεις η μέθοδος χωρίς να εμφανίζει τα αποτελέσματα στην οθόνη.

Στο τέλος με επαναληπτική δομή "for row in range (1, number_of_zeros):", δηλαδή τόσες επαναλήψεις όσα τα κενά κελιά η μέθοδος κάνει το επιλεγμένο συνεργατικό φιλτράρισμα και την επιλεγμένη πρόβλεψη για επιλεγμένο "k" και στοτέλος μηδενίζει τις λίστες "Similarity_value" και "Similarity_index" για να μην κάνει λάθως επιλογή γειτόνων από παλιές τιμές των 2 παραπάνω λιστών για τις νέες προβλέψεις.

Μέθοδος εμφάνισης των τελικών αποτελεσμάτων

Δημιούργησα μία νέα μέθοδο, την "delta(language):", η οποία εμφανίζει: α) την λίστα πριν την τροποποίηση της. β) εμφανίζει την λίστα μετά την τροποποίηση της. γ) εμφανίζει την απόκληση λάθους των προβλέψεων από την αρχική λίστα. δ) χρησιμοποεί αρχεία για την βέλτιστη εμφάνιση των αποτελεσμάτων.

Πιο συγκεκριμένα, η μέθοδος αρχικοποιείται με τις μεταβλητές: "error", "sum", "abs_sum", "file", οι οποίες υπολογίζουν το πλήθος των λανθασμένων προβλέψεων, το άθροισμα των αποκλήσεων των προβλέψεων από την αναμενόμενη τιμή, την απόλυτη τιμή του "sum" και τέλος το αρχείο.csv, δηλαδή το "Αποτελέσματα.csv" για την αποθήκευση των αποτελεσμάτων για καλύτερη εμφάνιση αντίστοιχα.

Στην συνέχεια εμφανίζει το μύνημα στην οθόνη και ταυτόχρονα το αποθηκεύει στο αρχείο. Μετά εμφανίζει την λίστα πριν την αφαίρεση των άγνωστων βαθμολογιών, δηλαδή την "number_grid2". Μετά κάνει την ίδια λειτουργία για την εμφάνιση της λίστας, δηλαδή την "number_grid" μετά την αφαίρεση των άγνωστων βαθμολογιών και την πρόβλεψη των βαθμολογιών από την "gamma(language):".

Έπειτα διαβάζει κάθε κελί των 2 λιστών με την δομή "for row in range(1, i+1): for col in range(1, j+1):" και συγκχρίνει την διαφορά της προβλευθήσας τιμής "number_grid[row][col]" με την αναμενόμενη τιμή "number_grid2[row][col]" για όλα τα κελιά (έτσι για απλότητα κώδικα, επειδή έσβησα τα "i_grid", "j_grid" για αποφυγή σφαλμάτων στην "gamma(language)") με 2 τρόπους. Ο ένας τρόπος είναι εάν η απόλυτη τιμή της διαφοράς της προβληφθήσας τιμής με την αναμενόμενη είναι κάτω από 0.1 λόγω των σφαλμάτων στρογγυλοποίησης και υλικού (επειδή η πράξη 0.1 + 0.2 = 0.3000000000000004 και όχι 0.3 λόγω της υλικής κατασκευής των υπολογιστών). Υπενθυμίζω ότι εδώ δεν αυξάνεται ο μετρητής "error" ούτε αθροίζει τις μικροδιαφορές στα "sum" και "abs_sum". Ο άλλος τρόπος είναι εάν η απόλυτη τιμή της διαφορά είναι μεγαλύτερη από το 0.1 τότε θεωρήται κανονική απόκληση.

Εδώ αυξάνονται οι μετρητές “error”, “sum” και “abs_sum”, όπου η 3η μεταβλητή αθροίζει την απόλυτη τιμή του σφάλματος και όχι την διαφορά. Υπενθυμίζω ότι και οι 2 τρόποι δεν εμφανίζουν τα αποτελέσματα στην οθόνη αλλά αποθηκεύουν αυτά στο αρχείο.

Στο τέλος κάνει την διαίρεση του “sum” με το “error” και αποθηκεύει το αποτέλεσμα στην “average_error” και την διαίρεση του “abs_sum” με το “error” και το αποθηκεύει στην “average_abs_error”. Αυτό γίνεται με την προυπόθεση ότι το “error > 0” για την αποφυγή σφάλματος εκτέλεσης λόγω της διαίρεσης του με το 0. Έτσι τα αποτελέσματα που εμφανίζονται στην οθόνη και στο αρχείο είναι το πλήθος των λανθασμένων προβλέψεων, το μέσο λάθος των προβλέψεων και το μέσο απόλυτο λάθος των προβλέψεων.

Τέλος Γ’ Ερώτημα

Πειράματα Που Έκανα Και Σύγκριση Των Μεθόδων

Συνοπτική εισαγωγή για το τελικό πείραμα
Αποφάσισα να παράξω μία λίστα με 40 χρήστες και 40 βιβλία και με αυτόν θα πειραματιστώ με την μέθοδο "gamma(language):" για να βρώ τον καλύτερο συνδυασμό μεθόδων: ομοιότητας μεταξύ χρηστών, προβλέψης βαθμολογίας και τιμών k-γειτόνων, Επίσης θα πειραματιστώ πως επιρρεάζεται το ποσοστό των άγνωστων βαθμολογιών του "number_grid" στην ακρίβια των προβλέψεων των βαθμολογιών. Πιο συγκεκριμένα στην αρχή θα επιλέξω ποσοστό 'αγνωστων βαθμολογιών 10% και με αυτό το ποσοστό θα επιλέξω όλον τον συνδυασμό των μεθόδων που της "gamma(language)" για "k = 3", "k = 6" και "k = 10". Όταν βρώ το καλύτερο αποτέλεσμα, δηλαδή αυτό με το μικρότερο μέσο απόλυτο λάθος κατά απόλυτη τιμή τότε με αυτές τις επιλογές που επέλεξα για το βέλτιστο τρόπο θα κάνω το ίδιο αλλά για 20% ποσοστό άγνωστων βαθμολογιών. Το κάθε πείραμα γίνεται 5 φορές. Υπενθυμίζω ότι κάθε φορά η λίστα γεμίζει με διαφορετικες τιμές λόγω της τυχαιότητας. Το ίδιο ισχύει και με το ποσοστό των άγνωστων βαθμολογιών λόγω τυχαιότητας.

Προετοιμεσία του Πειράματος
Έτσι το πλήθος των πειραμάτων εφμανίζεται παρακάτω:
4 ομοιότητες * 3 προβλέψεις * 3 k-γείτονες * 5 επαναλήψεις ανά πείραμα = 180 πειράματα για ποσοστό άγνωστων βαθμολογιών 10%.
Συν 10 επαναλήψεις * 1 την καλύτερη μέθοδο ομοιότητας * την καλύτερη πρόβλεψη επί 10 επαναλήψεις για ποσοστό άγνωστων βαθμολογιών 20%)). Αρα το σύνολο είναι 190 πειράματα.
Τα αποτελέσματα που θα εμφανίσω στον πίνακα 2 είναι:
α) Μέθοδος Ομοιότητας
β) Πρόβλεψη βαθμολογιών
γ) Τιμή k-γειτόνων
δ) Αριθμός άγνωστων βαθμολογιών
ε) Ποσοστό των άγνωστων βαθμολογιών
ζ) Αριθμός των λανθασμένων προβλέψεων
η) Μέσο απόλυτο λάθος κατά απόλυτη τιμή
θ) Ποσοστό διορθωμένων μεταβλητών
Όλα θα εμφανίζονται μετά από 5 επαναλήψεις διαιραιμενα με το 5 για εξοικονόμηση χρόνου. Εδώ θα εμφανιστούν μόνο τα μέσα όρη ανά πρόβλεψη βαθμολογιών, ανά μέθοδο ομοιότητα, ανά τιμή k-γειτόνων, η βέλτιστη επιλογή και τέλος το μέσο ποσοστό των άγνωστων βαθμολογιών που επιλέγει αρχικά η "gamma(language)".

Αποτελέσματα
Τα αποτελέσματα τα γράφω στους πίνακες και κώδικες. Πιο συγκεκριμένα στον Πίνακα 2. Εδώ θα εμφανιστούν αυτά που ανέφερα παραπάνω.

Μέσος όρος Πρόβλεψης 1	2,741745
Μέσος όρος Πρόβλεψης 2	2,777591667
Μέσος όρος Πρόβλεψης 3	2,728205
Μέσος όρος Jaccard	2,726906667
Μέσος όρος Cosine	2,757393333
Μέσος όρος Dice	2,765853333
Μέσος όρος Adjusted Cosine	2,746568889

Μέσος όρος για k = 3 2,854066667
Μέσος όρος για k = 6 2,721778333
Μέσος όρος για K= 10 2,671696667
Ο μέσος όρος για το 10% των άγνωστων βαθμολογιών = 0,099010417
Η βέλτιστη επιλογή είναι ο συνδυασμός Jaccard με Πρόβλεψη 3 και k = 6 και έχει μέσο απόλυτο λάθος κατά απόλυτη τιμή = 2,60092 αλλά το ποσοστό των επιτυχών προβλέψεων είναι = 1,4943%.
Επίσης ο συνδυασμός των μεθόδων που βγάζει το υψηλότερο ποσοστό επιτυχών προβλέψεων = 3,6616% είναι ο συνδυασμός Jaccard με πρόβλεψη 2 και k = 10 αλλά δυστυχώς βγάζει υψηλό μέσο απόλυτο λάθος = 2,73618.
Έτσι θα αποφασίσω να κάνω 10 επαναλήψεις με τον συνδυασμό μεθόδων: Jaccard, Πρόβλεψη 3 για k = 6 και ποσοστό βαθμολογιών = 20%. Πώς το έκανα: Πήγα στην γραμμή 593 και την εντολή "new = random.randint(0, 9)" την έκανα "new = random.randint(0, 4)" και έτσι το ποσοστό των άγνωστων βαθμολογιών είναι περίπου 20%. Τα αποτελέσματα φαίνονται παρακάτω:
Αριθμός άγνωστων μεταβλητών κατά μέσο όρο = 313,7
Ποσοστό άγνωστων μεταβλητών κατά μέσο όρο = 19,60625% περίπο 20%
Αριθμός λανθασμένων προβλέψεων κατά μέσο όρο = 307,7
Μέσο απόλυτο λάθος κατά απόλυτη τιμή μέοος όρος = 2,74643
Ποσοστό των επιτυχών προβλέψεων κατά μέοσ όρο = 1,9126%.

Απαντήσεις στις ερωτήσεις της εκτσιριροποίησης

1) Πώς επηρεάζει το ποσοστό των γνωστών βαθμολογιών την απόδοση (ακρίβεια) του συστήματος;

Απάντηση: Όσο αυξάνεται το ποσοστό των άγνωστων βαθμολογιών, τόσο αυξάνεται το ποσοστό των επιτυχών προβλέψεων. Δηλαδή στην βέλτιστη επιλογή που ανέφερα παραπάνω το ποσοστό των επιτυχών προβλέψεων για άγνωστες μεταβλετές 10% είναι 1,4943%. Με την αύξηση των άγνωστων μεταβλητών στο 20%, το ποσοστό ανεβαίνει στο 1,9126%, καλό αυτό. Δυστυχώς όσο αυξάνεται το ποσοστό των άγνωστων βαθμολογιών, τόσο το μέοσ απόλυτο λάθος αυξάνεται. Στην βέλτιση επιλογή το μέσο απόλυτο λάθος είναι 2,6. Με την αύξηση των άγνωστων μεταβλητών γίνεται 2,74. Άρα επιρρεάζει αρνητικά την απόδοση.

2) Ποια είναι η καλύτερη συνάρτηση ομοιότητας;

Απάντηση: Η καλύτερη συνάρτηση-μέθοδος ομοιότητας είναι η Jaccard, επειδή το μέσο απόλυτο λάθος είναι κατά μέσο όρο 2,726, ενώ οι άλλες μέθοδοι έχουν υψηλότερο μέσο απόλυτο λάθος. Το ίδιο ισχύει και με το ποσοστό των επιτυχών προβλέψεων. Η Jaccard έχει μέσο όρο 2,161%. που είναι το βέλτιστο. Ακολουθεί η Dice που έχει 0,021599354, μετά η Cosine που έχει έχει 2.099% και τέλος είναι η Adjusted Cosine που έχει 1.863%.

3) Πώς επηρεάζει το K την απόδοση (ακρίβεια) του συστήματος;

Απάντηση: Το k έχει θετική επίδραση στην ακρίβεια των αποτελεσμάτων. Δηλαδή όσο αυξάνεται το k τόσο πιο μικρό είναι το μέσο απόλυτο λάθος. Στο πείραμα που έκανα το μέσο απόλυτο λάθος για για k = 3 είναι 2,854, για για k = 6 είναι 2,721 και για k= 10 είναι 2,671696667. Το ίδιο ισχύει και για το ποσοστό των επιτυχών

προβλέψεων. Για k = 3, ο μέσος όρος του ποσοστού επιτυχών προβλέψεων είναι 2.018%. Για k = 6, το ποσοστό αυξάνεται σε 2,020% και για k = 10, το ποσοστό είναι 2,173%.

4) Ποια είναι η καλύτερη μέθοδος πρόβλεψης;

Απάντηση: Η καλύτερη μέθοδος πρόβλεψης ως προς αναφορά το μέσο απόλυτο λάθος είναι η Πρόβλεψη 3 με μέσο απόλυτο λάθος 2,728. Ακολουθούν μετά η Πρόβλεψη 2 και η Πρόβλεψη 1. Η καλύτερη μέθοδος πρόβλεψης ως προς το ποσοστό των επιτυχών προβλέψεων είναι Πρόβλεψη 2 με ποσοστό επιτυχών βαθμολογιών 2,144%. Ακολουθεί η Πρόβλεψη 3 με ποσοστό 2,077% και τέλος η Πρόβλεψη 1 με ποσοστό 1,99%.

Bonus: Σχεδιάστε μια δική σας μέθοδο πρόβλεψης πέρα από τον απλό μέσο όρο και τον σταθμισμένο μέσο όρο, που πειραματικά θα δείξετε ότι έχει τουλάχιστον αντίστοιχη απόδοση με τις άλλες δύο

Απάντηση: Η Πρόβλεψη 1 με συριακούς γείτονες που έχει καλύτερο μέσο απόλυτο λάθος σε σχέση με την Πρόβλεψη 2 αλλά χειρότερο από την Πρόβλεψη 3. Το ποσοστό των επιτυχών προβλέψεων είναι το χειρότερο. Το θετικό είναι ότι είναι γρήγορη στον υπολογισμό από γραμμές κώδικα και αριθμό επαναλήψεων. Επίσης είναι ανεξάρτηση από συνεργατικά φιλτραρίσματα. Δηλαδή με λίγα λόγια δεν έκανα 5 φορές Πρόβλεψη 1 σε κάθε περίπτωση αλλά 20 φορές λόγω του ότι δεν εξαρτάται με τις μεθόδους ομοιότητας μεταξύ χρυστών. Επίσης η αραβική γλώσσα είναι μία καινοτομία για αστείο και στο παράρτημα, ο πίνακας 3 εξηγεί πώς προφέρονται τα αραβικά γράμματα με την δική μου διάλεκτο, την αλαβική. Υπενθυμίζω ότι διαβάζονται από αριστερά προς τα δεξιά.

Τέλος των πειραμάτων που έκανα και της σύγκριση των μεθόδων

ΠΙΝΑΚΕΣ ΚΑΙ ΚΩΔΙΚΕΣ

ΠΙΝΑΚΑΣ 1

	0	1	2	3	i
0	'Χρήστης hid \ Βιβλίο hid'	'Βιβλίο 1'	'Βιβλίο 2'	'Βιβλίο 3']	
1	'Χρήστης 1'	1	2	3]	
2	'Χρήστης 2'	4	5	6]	
3	'Χρήστης 3'	7	8	9]	
j					

ΚΩΔΙΚΑΣ 1

```
def add_2_greek(i, j):
    print("Προσθήκη στηλών")
    A = 0
    for B in range (1, i):
        for row in number_grid:
            if A == i + 1:
                break
            if A == 0:
                aa = 'Βιβλίο '
                bb = str(j+1)
                cc = aa+bb
                row.append(cc)
            if A > 0:
                print("Βάλε βαθμολογία στην στήλη", B, "και γραμμή", A)
                new = input()
                new = int(new)
                row.append(new)
            A = A + 1
        j = j + 1
```

Εικόνα 1: Η Χάιντι



Πίνακας 2. Τα αποτελέσματα του πειράματος που έκανα.

Μέθοδος Ομοιότητας	Μέθοδος Πρόβλεψης	Τιμή k-γειτόνων	Αριθμός Άγνωστων Βαθμολογιών	Ποσοστό Άγνωστων Βαθμολογιών	Αριθμός Λανθασμένων Προβλέψεων	Μέσο απόλυτο λάθος	Ποσοστό διορθωμένων μεταβλητών
Jaccard	Πρόβλεψη 1	3	162	10,13%	159,2	2,80408	0,017283951
Jaccard	Πρόβλεψη 2	3	149,8	9,36%	147,2	2,84452	0,017356475
Jaccard	Πρόβλεψη 3	3	159,8	9,99%	156,4	2,7895	0,021276596
Jaccard	Πρόβλεψη 1	6	158,2	9,89%	154,4	2,70098	0,024020228
Jaccard	Πρόβλεψη 2	6	160,4	10,03%	156,8	2,73436	0,02244389
Jaccard	Πρόβλεψη 3	6	160,6	10,04%	158,2	2,60092	0,01494396
Jaccard	Πρόβλεψη 1	10	156,8	9,80%	153,6	2,65796	0,020408163
Jaccard	Πρόβλεψη 2	10	158,4	9,90%	152,6	2,73618	0,036616162
Jaccard	Πρόβλεψη 3	10	158,8	9,93%	155,6	2,67366	0,020151134
Cosine	Πρόβλεψη 1	3	158,4	9,90%	154,6	2,92804	0,023989899
Cosine	Πρόβλεψη 2	3	154,4	9,65%	149,8	2,8054	0,029792746
Cosine	Πρόβλεψη 3	3	153	9,56%	150	2,9627	0,019607843
Cosine	Πρόβλεψη 1	6	157,2	9,83%	154	2,7415	0,020356234
Cosine	Πρόβλεψη 2	6	164,4	10,28%	162,8	2,74332	0,00973236
Cosine	Πρόβλεψη 3	6	162,6	10,16%	159,8	2,70104	0,017220172
Cosine	Πρόβλεψη 1	10	161,8	10,11%	158,8	2,63608	0,018541409
Cosine	Πρόβλεψη 2	10	162,8	10,18%	159,2	2,67366	0,022113022
Cosine	Πρόβλεψη 3	10	166,8	10,43%	162,2	2,6248	0,027577938
Dice	Πρόβλεψη 1	3	162,4	10,15%	158,4	2,88388	0,024630542
Dice	Πρόβλεψη 2	3	165,6	10,35%	162,2	3,01352	0,020531401
Dice	Πρόβλεψη 3	3	155,4	9,71%	152,8	2,79428	0,016731017
Dice	Πρόβλεψη 1	6	159,6	9,98%	155,6	2,68096	0,025062657
Dice	Πρόβλεψη 2	6	152,8	9,55%	149,8	2,72582	0,019633508
Dice	Πρόβλεψη 3	6	152	9,50%	147,2	2,75972	0,031578947
Dice	Πρόβλεψη 1	10	151,6	9,48%	149	2,63358	0,017150396
Dice	Πρόβλεψη 2	10	162	10,13%	159,2	2,6925	0,017283951
Dice	Πρόβλεψη 3	10	165,2	10,33%	161,6	2,70842	0,021791768
Adjusted Cosine	Πρόβλεψη 1	3	160,6	10,04%	158	2,83486	0,01618929
Adjusted Cosine	Πρόβλεψη 2	3	152,2	9,51%	149,2	2,80606	0,019710907
Adjusted Cosine	Πρόβλεψη 3	3	158,6	9,91%	156,2	2,78196	0,015132409
Adjusted Cosine	Πρόβλεψη 1	6	160,4	10,03%	158,4	2,7578	0,012468828
Adjusted Cosine	Πρόβλεψη 2	6	159	9,94%	156	2,81942	0,018867925
Adjusted Cosine	Πρόβλεψη 3	6	152,8	9,55%	148,8	2,6955	0,02617801
Adjusted Cosine	Πρόβλεψη 1	10	159,6	9,98%	156,6	2,64122	0,018796992
Adjusted Cosine	Πρόβλεψη 2	10	154,6	9,66%	151	2,73634	0,023285899
Adjusted Cosine	Πρόβλεψη 3	10	152,4	9,53%	149,8	2,64596	0,017060367
Jaccard	Πρόβλεψη 3	6					

Πίνακας 3. Το αραβικό αλφάβητο και η προφορά των γραμμάτων.

Ελεύθερα Γράμματα		Ενωμένα Γράμματα		Ενωμένα Γράμματα	
Γράμμα	Προφορά	Γράμμα	Προφορά	Γράμμα	Προφορά
ا	I	ـا	L	ـﺜ	θ
ب	Y	ـبـ	u ή ui	ـﻫ	φ
ت	Ÿ	ـبـ	u ή ui	ـيـ	u ή ui
ث	Ÿ	ـتـ	ï ή uï	ـيـ	u ή ui
ج	τθ	ـتـ	ï ή uï	ـلا	K
ح	τθ	ـثـ	ï ή uï	ـد	u ή ui
خ	τθ	ـثـ	ï ή uï	ـد	u ή ui
د	jz	ـجـ	τθ ή π	ـئ	u ή ui
ذ	jz	ـجـ	d	ـئ	u ή ui
ر	j	ـحـ	τθ ή π	ـة	a
ز	j	ـحـ	d	Παρατηρήσεις	
س	djzıı	ـخـ	τθ ή π	Υπενθυμίζω ότι τα γράμματα με το κόκκινο ή ανήκουν στην κατηγορία "u". Τα γράμματα με το πράσινο ή ανήκουν στην κατηγορία "i". Τα γράμματα με το μπλέ ή ανήκουν στην κατηγορία "τθ". Ο κανόνας για τα γράμματα της κατηγορίας "u" είναι: Όταν υπάρχει μόνο 1 γράμμα και δεν ακολουθούν στην σειρά άλλα γράμματα της ίδιας κατηγορίας, της κατηγορίας "i", σύνθετα γράμματα που περιέχουν "i" και "ui" και γράμματα της κατηγορίας "Y" τότε προφέρεται "u". Για παράδειγμα η λέξη بلبل προφέρεται "Julu". Εάν υπάρχει έστω και ένα γράμμα των κατηγοριών που ανέφερα τότε το γράμμα προφέρεται "ui". Για παράδειγμα η λέξη لبب προφέρεται "Yuıt". Ο κανόνας για τα γράμματα της κατηγορίας "i" είναι ίδιος με της κατηγορίας "u". Εξαίρεση υπάρχει στην κατάληξη κάποιων λέξεων όπου παρόλου που υπάρχει 1 μόνο γράμμα αυτής της κατηγορίας η προφορά είναι "ui" για λόγους ευφωνίας. Για παράδειγμα η λέξη تَريد προφέρεται "jzujuï" και όχι "jzujï".	
ش	djzïi	ـخـ	d		
ص	djzO	ـسـ	uıuıı		
ض	djzÓ	ـشـ	uıuïi		
ط	b	ـصـ	ıO		
ظ	b	ـضـ	ıÓ		
ع	ετ	ـعـ	Ωτθ		
غ	έτ	ـعـ	Ω		
ف	uó	ـعـ	c		
ق	üg	ـغـ	Ώτθ		
ك	Js	ـغـ	Ώ		
ل	J	ـغـ	c		
م	p	ـفـ	ó		
ن	djz	ـفـ	úg		
ه	o	ـقـ	o		
و	g	ـقـ	üg		
ي	sh	ـكـ	S		
لا	Ψ	ـلـ	I		
ى	sh	ـلـ	t		
ة	o	ـمـ	o		
ء	c	ـنـ	i ή ui		
ؤ	g	ـنـ	i ή ui		
ئ	sh	ـهـ	a		

Ο κανόνας για τα γράμματα της κατηγορίας "τθ" είναι ότι όλες οι λέξεις προφέρονται με "τθ". Η επιλογή "π" γίνεται μόνο και μόνο για αστείου λόγους και δεν υπάρχει κανόνας για αυτή την επιλογή. Για παράδειγμα η λέξη كه نامكح προφέρεται και "τθSoLi αS" και "πSoLi αS".